Cross-Layer Observability for LLM-Assisted Test Automation: A Reference Architecture and Web Feasibility Study

Author: Suneet Malhotra **ORCID:** 0009-0003-8707-9590 **Affiliation:** Independent researcher in AI-augmented test automation. Author is also Senior Manager, Test Engineering at Motorola Solutions; this work is independent of that role and uses only public infrastructure.

> *First-page footnote.* The implementation and the §6.1 evaluation were developed independently with public infrastructure and a public reference application (a TodoMVC web demo). The three-tier execution layer is target-agnostic in design — it supports web, mobile, and hardware-in-the-loop instantiations — but only the web modality is evaluated in this article; mobile and hardware-in-the-loop are described as architectural extensions and are not evaluated. Practitioner observations in §1 are drawn from 16+ years of test-automation work and are used as context, not employer evidence; no proprietary systems, products, code, data, screenshots, or operational metrics are described.

Prepared for: Journal of Systems and Software (Elsevier) — In Practice scope (Applied Research Report).
Contact: suneetmalhotra2002@gmail.com · https://suneetmalhotra.com **Companion code:** https://github.com/SuneetMalhotra/agent-harness (MIT-licensed; reproducibility manifest at `ARTIFACTS.md`).
Figures: Figures 1 (three-layer architecture, §2), 2 (five-agent SDLC pipeline, §4), and 3 (self-healing locator cascade, §5.2) appear inline as flowchart diagrams.

Abstract

Test automation built with LLM-based agents spans three partially connected layers — agent-authored frameworks, multi-agent SDLC pipelines, and self-healing execution on heterogeneous hardware — that existing systems treat in isolation. This article describes an *agent harness*: a reference architecture that keeps the three layers separate but connects them through a shared observability substrate — a schema-defined event store with a query API, designed so each layer's agent prompts can be conditioned on events emitted by the others. In the reference implementation, a coding agent authors a TypeScript framework under human review, a five-agent pipeline (PM, QA, Automation Engineer, Developer, PR Reviewer) handles design-to-test over the Model Context Protocol, and a three-tier execution plane spans a physical bench, a cloud device farm, and ephemeral virtual hardware. We report a small public web feasibility study whose purpose is to establish that the substrate is wired and queryable end-to-end — not that it improves agent decisions, which a prompt-level A/B (identified here as the key next step) would be needed to show. A single live run on a public TodoMVC demo (Claude Sonnet 4.6, 30 test cases) serves as a controlled calibration environment: it recovers 29 of 30 injected locator failures, and a vision-based visual-assertion judge reaches a preliminary Cohen's $\kappa = 0.667$ against 24 seeded screenshots (95% CI 0.47–0.87), pending the human-rater audit shipped with the repository. On one complex public React application (Supabase Studio; 56 perturbations, one model family), an adversarial locator-perturbation benchmark with an identity oracle shows the LLM DOM-healer recovers the correct element in 63% of cases — against 34% for a text heuristic, 21% for the self-healing-Selenium tool Healenium, and 0% for the broken selector — and is the only resolver robust to DOM restructuring; at a 27% false-heal rate and ~5 s per heal it is promising but not yet safe for unsupervised CI. Mobile and hardware-in-the-loop are instantiations of the target-agnostic design but are not evaluated here. All code, data, and audit packets are public.

---

1. Introduction

In production engineering practice, end-to-end test automation is rarely a single-layer concern, regardless of whether the system under test is a mobile app, a web application, or a hardware-in-the-loop assembly: a framework has to be authored, operated, and executed; tests must run on a mix of simulators, cloud devices, headless browsers, and physical benches; and the whole must integrate with CI, defect tracking, test management, and design tooling. The question has shifted from “can an agent contribute to a layer?” to “what changes when the layers couple?” Three failure modes recur when the layers run in isolation: framework refactors against hand-curated priorities rather than execution data; pipelines write tests against routing tags that go stale; execution layers collect observability data nobody reads.

The 2023–2026 literature on *agentic software engineering* — the umbrella term Hassan et al. [27] use for SE 3.0, in which intelligent agents pursue complex SE goals rather than single-shot code generation — breaks into three largely independent lanes. Self-healing locators have an open-source antecedent in Healenium [16], which uses tree-edit distance over rendered DOM trees. LLM-guided mobile GUI exploration has its strongest published instance in GPTDroid [17], reporting approximately 32 percent activity-coverage improvement and 31 percent more bugs on 93 Google Play apps; AutoDroid [19] and AppAgent [20] extend the LLM-driven mobile-agent line to general task automation, LELANTE [18] couples LLM-driven action selection with an Android execution pipeline, and Liu et al.’s *Seeing-is-Believing* [28] introduces multi-agent vision-driven non-crash functional bug detection on mobile GUIs — the closest published antecedent to the §6.1 visual-assertion service, on a different modality. Each treats one execution-layer concern; none couples to a multi-agent pipeline or to framework authoring. Multi-agent SDLC pipelines have closely related antecedents in MetaGPT [23] and ChatDev [26], which decompose software development across PM, Engineer, QA, and Reviewer roles communicating either through structured artifacts in shared in-memory state (MetaGPT) or through chat-mediated communicative-dehallucination dialogues (ChatDev); §2 and §4 detail how the substrate, execution-tier integration, and cross-layer telemetry feedback differ. Hou et al. [3] identify tool integration and end-to-end traceability as recurring gaps in LLM-for-SE deployments; the substrate described here is one concrete answer. Fan and Harman [4] frame the open problem set around hallucination, evaluation, and the lack of grounded execution feedback.

This article describes a coupling pattern I call an *agent harness*. The term has recent independent lineage: Meng et al. [15] formalize the harness as a labeled-transition-system wrapping a single agent’s execution loop. The term as used here is a domain-specific instantiation applying the same surfacing principle one level up: a *cross-layer* coupling across three agent-augmented layers that share an observability substrate. Framework refactoring is driven by per-test latency and healing-rate data from execution; pipeline routing decisions are driven by per-tier flake rates. Each layer remains testable in isolation; the coupling is a thin data substrate, not a new monolith. §2.1 provides a worked instantiation in which the QA agent receives a structured healing digest from the previous run and produces a coverage recommendation the authoring agent can act on.

The contribution is the coupling. Against [15], scope (three-layer SDLC, not single-agent infrastructure). Against MetaGPT [23] and ChatDev [26], substrate (typed-event MCP handoffs with execution telemetry routed back into the authoring layer, not shared in-memory state or chat-mediated communicative-dehallucination loops). Against GPTDroid [17], AutoDroid [19], AppAgent [20], LELANTE [18], level (framework-level authoring plus multi-tier execution, not exploration-driven GUI testing). Against Healenium [16], method (cache-primary, LLM-DOM-healer secondary, vision-fallback tertiary). Against the surveyed multi-agent SE literature: HPCAgentTester [7] generates HPC unit tests via a Recipe/Test-Agent critique

loop but addresses no execution-tier heterogeneity; ALMAS [8] is an agile-role pipeline operating on a single execution context with no cross-tier routing substrate; Chia et al. [9] catalogue pipeline contributions but do not enumerate cross-layer observability as a category.

The article makes three contributions: (1) the *agent harness* reference architecture — three layers connected by a shared, schema-defined observability substrate designed to expose cross-layer signals to each layer's prompts; (2) the *reflexive-correctness* sub-contribution of §3.2, making explicit — and offering a layered, auditable answer to — the question of how an LLM-assisted testing framework is itself known to test the product correctly; and (3) a public web evaluation — a TodoMVC calibration run plus an adversarial locator-healing benchmark on Supabase Studio with ground truth and baselines — establishing that the substrate is wired and queryable end-to-end and that LLM-based healing materially outperforms a brittle selector, a text heuristic, and the industry self-healing tool Healenium (63% correct-element recovery vs 0% / 34% / 21%) on a complex live UI. We are deliberately explicit about what is *not* yet shown: that exposing the cross-layer digest measurably changes agent decisions is a design objective, not a demonstrated result, and the prompt-level A/B that would test it is identified as the priority follow-up (§7.3). §6.1 reports a single live web feasibility run on a public TodoMVC application, used as a controlled calibration environment (30 test cases, 29/30 combined locator recovery, a preliminary visual-assertion $\kappa = 0.667$ on a 24-image seeded corpus). Mobile and hardware-in-the-loop are instantiations of the target-agnostic design and are not evaluated in this article.

The remainder describes the pattern (§2), authoring (§3), operating (§4), execution (§5), evaluation (§6), and adoption (§7).

---

2. The agent harness pattern

The harness has three layers and one data substrate.

Authoring layer. An LLM coding assistant (the reference implementation uses a hosted-model CLI; the architecture is provider-agnostic, with open-weights local-model backends such as Ollama-served Llama-family models supported equally — consistent with open-weights LLM-based testing work in [24]) proposes changes under human review. The prompting style follows the chain-of-thought tradition [5]; the workflow follows the coding-agent adoption literature [12].

Operating layer. A five-agent SDLC pipeline: Product Manager, QA Engineer, Automation Engineer, Developer (optional), Pull Request Reviewer. Agents share no state directly; each writes its output to an external artifact system that the next reads as input, with contracts tuned to SDLC artifacts (PRDs, test plans, code, review comments). The artifact-mediated handoff echoes multi-agent pipelines in adjacent domains such as automated penetration testing [6]. The handoff substrate is the Model Context Protocol [11]. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26]; the substrate is not. MetaGPT communicates through shared in-memory state inside one process; ChatDev uses chat-mediated communicative-dehallucination dialogues to negotiate hallucination drift between roles; this pipeline writes typed artifacts across MCP server boundaries (filesystem, Git, Jira, Confluence) with execution telemetry routed back into the authoring layer's prompt context. Neither MetaGPT nor ChatDev exposes an equivalent layer for runtime execution-failure recovery; the §5.2 three-tier healing cascade (cache → DOM → vision) is the differentiator from multi-agent SDLC frameworks, which terminate at code generation and PR review without an execution-time self-healing substrate.

Execution layer. Three hardware tiers, instantiated per target modality. Tier 1: a physical bench — mobile device bench for real BLE peripherals, cellular radio, and sensor state; local browser instance for web;

physical fixture rig for hardware-in-the-loop. Tier 2: a commercial cloud farm — real-device farm for mobile cross-OS coverage, cross-browser farm (BrowserStack, Sauce Labs, etc.) for web, vendor lab access for hardware. Tier 3: ephemeral virtual hardware emulating back-end peripherals, mock services, or sensor injectors for end-to-end paths. An intelligence layer sits between test code and the underlying target: cached semantic locators, an LLM healer on cache miss, a vision fallback when DOM healing fails, and a multimodal visual assertion service.

The observability substrate. All three layers emit into a shared store. The canonical schema is in `types.ts`; load-bearing types are `ObservabilityEntry`, `HealingEvent`, `AssertionEvent`, and `AgentHandoff`, each keyed to a test case, commit, and agent. A thin query API exposes per-test and per-suite aggregates to prompts as structured tables. These keys let the QA and authoring agents change behavior in response to execution data. Unlike general-purpose distributed tracing (for example, OpenTelemetry) or agent-framework run logs (such as LangGraph or AutoGen traces), which record events primarily for post-hoc human inspection, the substrate is designed to be read back *in-loop*: a digest of one layer's events is injected into another layer's prompt. The contribution is this cross-layer prompt-conditioning contract and its schema — not the act of recording events; whether agents reliably act on the digest is the open question §6.1 and §7.3 address.

What distinguishes this from three integrated systems is that each layer's prioritization decisions are *designed to be conditioned on* observability events emitted by the other layers, not on authoring-time guesses. The substrate is keyed to enable per-layer agent decisions to be conditioned on cross-layer telemetry: §6.1 reports the substrate's measured dispatch and recovery behavior on a live run, and §7.3 queues the prompt-level A/B that isolates the behavioral-change claim. Where Meng et al. [15] propose the agent harness as a wrapping LTS around *one* agent, this pattern exposes a *shared* event stream across three agent-augmented layers.

2.1 An intended usage scenario

The following walks through the substrate's intended use; §6.1 confirms the digest is queryable from agent prompts and reports the underlying healing-event distribution, and §6.3 and §7.3 record the deferred prompt-level A/B. The §6.1 live-LLM reference run produced 30 `executing.healing` events: 0 resolved from the pre-warmed cache (the warmup keys did not match the live agent-generated test-case IDs — a finding to reproduce on a stable test-case corpus before claiming a cache-hit rate), 11 from `dom-healer` (9 at Tier 1, 2 at Tier 2), 18 from `vision-healer` (9 at Tier 1, 5 at Tier 2, 4 at Tier 3), and 1 unrecoverable failure (`TC-007`, Tier 2). In the intended usage, the QA Engineer agent's coverage-prioritization prompt is fed a digest filtered to `kind=healing` and grouped by `resolvedStrategy`: *"0/30 cache, 11/30 dom-healer, 18/30 vision-healer, 1/30 unrecoverable (TC-007 tier2). Vision-fallback usage at 60% suggests locator instability; recommend stabilizing locators on Tier-1 surfaces and re-warming the cache against the new test-case IDs."* The QA agent's next-coverage output preserves the recommendation as a structured field in the test-case ticket; the Authoring agent, when next asked to refactor the resolver, receives the same digest. The substrate carries the digest; the open question of whether agents reliably act on it under varying digest content is a §7.3 follow-up.

Figure 1 shows the three layers and substrate.

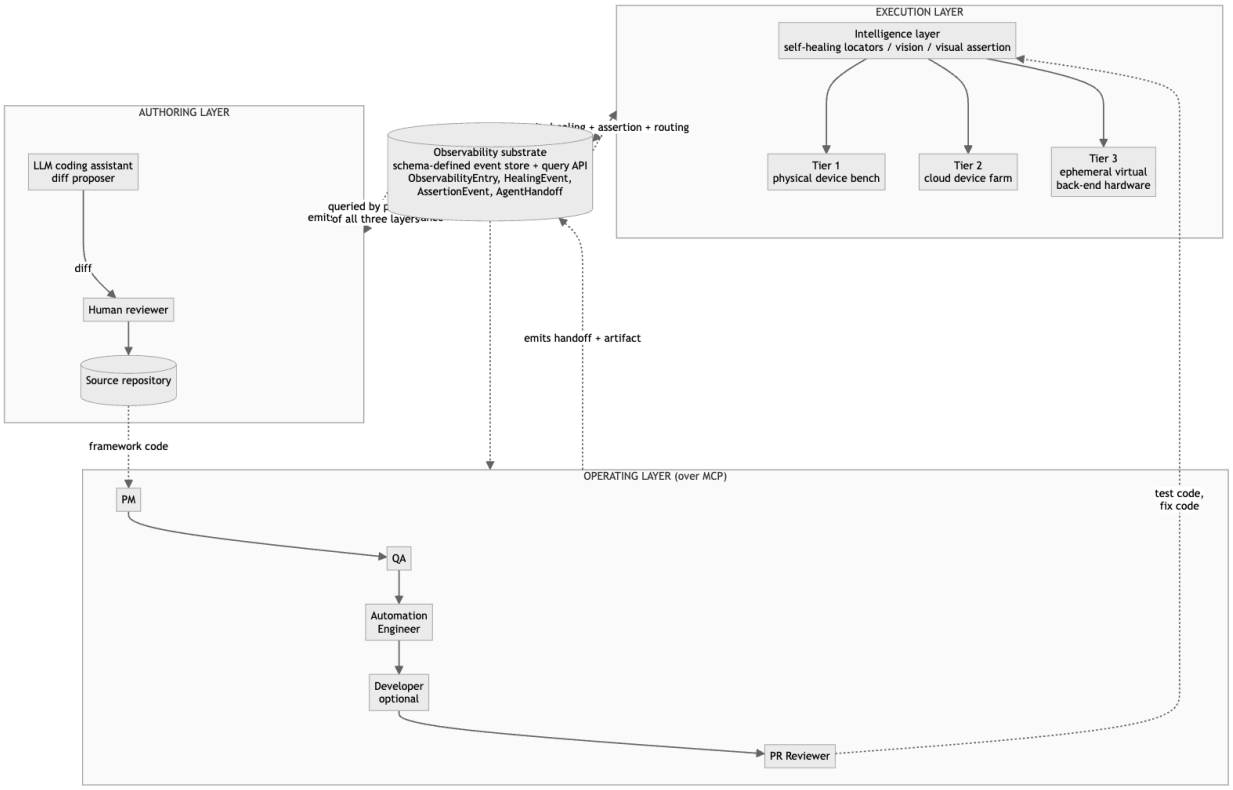


Fig. 1. Three-layer agent harness architecture. The authoring layer produces framework code; the operating layer (five-agent SDLC pipeline over MCP) produces test code and fixes; the execution layer routes to one of three hardware tiers via an intelligence layer that mediates locator resolution, visual assertion, and observability emission. The dashed-line **observability substrate** at the bottom is the coupling: every layer emits typed events into it (`ObservabilityEntry`, `HealingEvent`, `AssertionEvent`, `AgentHandoff` defined in `types.ts`), and prompts in every layer query it as structured tables. The contribution is the substrate-mediated coupling, not any single layer.

---

3. The authoring layer

The framework's TypeScript codebase was developed over roughly five months with substantial LLM coding-assistant support: an engineer writes a prompt, the assistant proposes a diff, the engineer reviews and merges. The assistant is invoked via a provider-flexible CLI wrapper (the reference implementation supports a hosted-model OAuth path and a local-weight path through Ollama, similar to the open-weights LLM-testing approach demonstrated in [24] at `https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing`); the architectural pattern does not depend on which model backend is chosen. Practitioner observations from a single deployment; counter-evidence on LLM coding limits [14] and longitudinal Copilot studies [12] suggest the velocity direction is plausible but the magnitude does not generalize.

Practitioner observation (not an empirical result). Five agent-assisted framework modules had prompt-to-merge wall-clock times of approximately 1.5, 2.0, 2.5, 3.0, and 4.5 hours; three hand-authored modules had approximately 6, 8, and 11 hours. With N=5 and N=3 on a single workstation, these are anecdotal observations, not a powered speedup estimate. Consistent with [14], the slowest agent-assisted modules

involved complex concurrency or hardware-interaction logic. Reported here rather than in §6.1 because the sample carries no inferential weight.

3.1 Provenance

The provenance discipline the framework adopts is conceptual: a commit-message convention identifying authorship class (LLM-authored, LLM-authored-then-revised, or human-authored) and a pull-request convention linking to the prompt and reasoning trace when exposed. At any line, an engineer can in principle answer “who wrote this, from what prompt?” with a defined chain of evidence. The discipline is necessary because, without it, the path from a wrong line back to the prompt that produced it is not recoverable; it is independent of which model backend is in use.

3.2 The reflexive-correctness question

An LLM-assisted framework used to test product code raises the *reflexive-correctness* problem: how is the framework itself known to test the product correctly? The conventional answer (“tests test the framework”) is circular when the tests are also LLM-authored. The reflexive-correctness problem is the test oracle problem [21] in a new form: an oracle whose generation process is itself the property under test.

The answer here is *layered validation* — the harness does not formally close the loop, but it makes correctness *auditable* through three independent oracle sources, each operating on a layer the LLM cannot reflexively validate:

- **Hand-authored unit tests on framework library code,** outside the LLM-assisted authoring pipeline. These tests are intentionally not regenerated when the library is refactored; they pin behavioral invariants the assistant might otherwise drift past. Connects to the structural testing of LLM agents in [2].
- **Tier-1 hardware ground truth on product-level tests.** When a test on a physical device observes a hardware-level outcome the simulator-only path cannot fake (a BLE characteristic value, a sensor reading), that observation is a non-LLM oracle independent of both the framework and the product LLM-generation chain.
- **A Pull Request Reviewer agent (§4) as a final structured-rubric gate,** with explicit rubric items that it (a) is a different LLM call than the one that authored the change, and (b) must justify its disposition with citations to the diff, not the prompt that produced the diff.

The framing as a named sub-contribution matters because the LLM-for-SE literature [3, 4] treats agent-authored testing as a generation problem (better tests, more coverage) and rarely confronts the reflexive correctness of the agent-authored testing infrastructure itself — a gap empirically confirmed by Hasan et al.’s [29] study of testing practices in open-source AI agent frameworks, which finds developers concentrate on deterministic components and systematically under-test foundation-model behavior; the layered-validation approach is one concrete answer, and the open problem of *formal* reflexive correctness (a closed-loop guarantee that an LLM-assisted framework is sound on the property it tests) is identified as future work for the LLM-for-SE community.

---

4. The operating layer: a five-agent SDLC pipeline

Each agent is a Markdown specification (a “skill” or “agent prompt”) plus access to a defined set of MCP servers. The Product Manager turns a design artifact or stakeholder description into a PRD and acceptance criteria (the elicitation prompt draws on [13]). The QA Engineer turns a PRD into a test specification with

traceability links. The Automation Engineer turns test cases into WebDriverIO test files and a pull request. The Developer (optional) implements feature code. The Pull Request Reviewer emits a structured review with a disposition (approve, request changes, block). Agent specifications and I/O contracts live in `agents/`. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26], and to the unit-test-focused multi-agent consensus approach of Xu et al. [30] (an actor/critic-style hallucination-to-consensus loop on JUnit generation); the MCP-substrate architecture differs from MetaGPT's shared in-memory state, ChatDev's chat-mediated dialogue, and Xu et al.'s consensus-prompt loop in that execution-tier telemetry routes back into the authoring layer's prompt context (§2 substrate). A like-for-like A/B against any of the three baselines is queued at §7.3. The pipeline runs end-to-end or step-by-step; end-to-end handles low-complexity features, step-by-step when intermediate review is valuable.

MCP [11] replaces per-tool custom adapters with a typed interface per server. Each agent's output carries a structured trailer recording agent identifier, model version, prompt, input artifact IDs, and timestamp. The path from a downstream defect back through test code, test case, PRD, and design artifact is queryable; I call this *inter-agent provenance*, distinct from the intra-agent provenance of §3.1.

The stub-provider end-to-end run completes in sub-second wall clock — confirmation that the five agents hand off cleanly over MCP, not a measurement of live-LLM pipeline latency (§6.1 expands on this distinction). Step-by-step runs with the live-LLM path range from minutes (small features) to several hours (cross-functional features with multiple revision cycles). In the author's prior experience across several engineering contexts, comparable hand-authored scenarios took multiple working days; a practitioner recollection, not a measured distribution. A defensible economic comparison would include the cost of the agent infrastructure, which is out of scope (§7.2). The dominant failure mode was upstream specification gaps (acceptance criteria left implicit at the PRD stage) rather than agent-internal errors; the elicitation work in [1] addresses this gap directly. Figure 2 shows the pipeline and MCP servers.

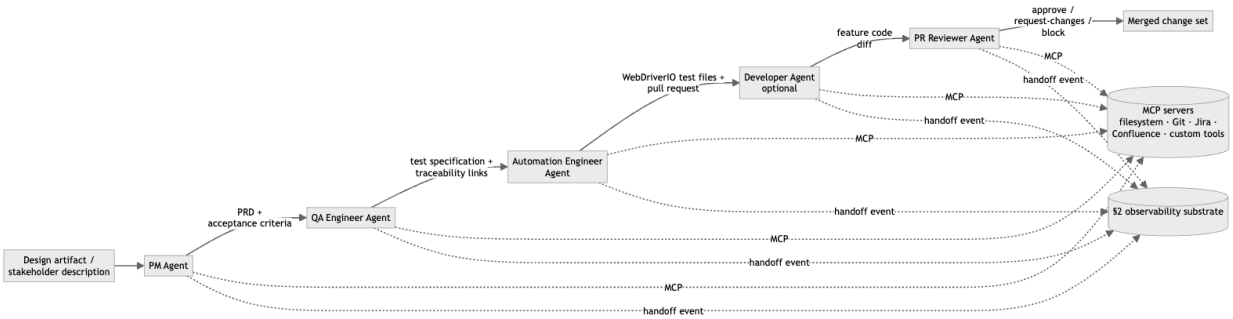


Fig. 2. Five-agent SDLC pipeline. Each agent is a Markdown specification (a "skill" or "agent prompt") plus access to a defined set of MCP servers; agents share no in-process state. Each agent's output carries a structured trailer (agent identifier, model version, prompt, input artifact IDs, timestamp) so a downstream defect is traceable back through test code, test case, PRD, and design artifact — the *inter-agent provenance* claim of §4. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26]; the differentiating substrate is the MCP-mediated typed handoff and the §2 observability substrate that lets execution-tier telemetry route back into the authoring layer.

4.1 LLM provider backends

Each agent calls the same `ModelProvider` interface (`generate(system, user, responseFormat, temperature) → Promise<string>`). The harness ships three live backends, exercised end-to-end against the §2 observability substrate: a hosted-frontier path (Anthropic Claude via Claude OAuth), an open-weights

local path (Ollama-served Llama-family models), and a deterministic offline stub (for CI and reviewer reproduction). Backend selection is a single CLI flag (`--provider <name>`).

Backend	Status	Endpoint	Weights	Default model	Credential	Use case
`stub`	**live**	in-process	n/a	n/a	none	Deterministic offline reproduction; CI; demos without network
`anthropic`	**live**	`claude -p` subprocess (OAuth)	hosted-frontier	`claude-sonnet-4-6`	Claude OAuth session	Reference live-LLM path used in §6.1 walkthrough
`ollama`	**live**	`http://localhost:11434/api/chat`	**open-weights (local)**	`llama3.2`	none	Open-weights replication; air-gapped deployment; fine-tuned variants

The `ollama` adapter routes the five agents through a locally running Ollama server and accepts any model Ollama can serve (Llama 3.x, Mistral, Qwen, Gemma — anything compatible with the Ollama runtime). It is the open-weights complement to the hosted-frontier path (`anthropic`) and follows the open-weights LLM-testing pattern that Rehan et al. [24] demonstrated for fine-tuned Llama-2-7b on focal-method-to-test-case generation (`https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing`). The relevant adoption tradeoffs: hosted-frontier providers have stronger calibration and reasoning depth at per-call cost and outbound data transfer; the open-weights local path eliminates both at the cost of fine-tuning effort or larger-model latency. Because the §2 observability substrate is keyed on the agent identifier and the model version rather than on the provider, swapping live backends does not invalidate the coupling — a §7.2 study comparing `anthropic` against `ollama` already runs end-to-end against the substrate. The `ModelProvider` interface is provider-flexible by construction: additional hosted-frontier backends (OpenAI, Gemini) require only a `generate()` implementation against the documented HTTPS contract.

---

5. The execution layer: three hardware tiers

5.1 Tiers and routing

Tier 1 is a physical bench, the only tier that exercises tests requiring real hardware state inaccessible from emulators or headless browsers: BLE peripherals, cellular radio, biometric sensors, or hardware fixtures for embedded/IoT targets, as well as local browser instances when the test surface includes browser-specific behaviors. The reference deployment uses Android devices connected over USB to a remote Linux server addressable via ADB for the mobile case; web targets use a local Chromium instance driven by WebDriverIO; hardware-in-the-loop targets use a fixture rig with USB-controlled probes. Tier 2 is a commercial cloud farm driven through a uniform driver interface (Appium-compatible for mobile, WebDriver-compatible for web). Tier 3 is ephemeral virtual hardware emulating back-end peripherals, mock services, or

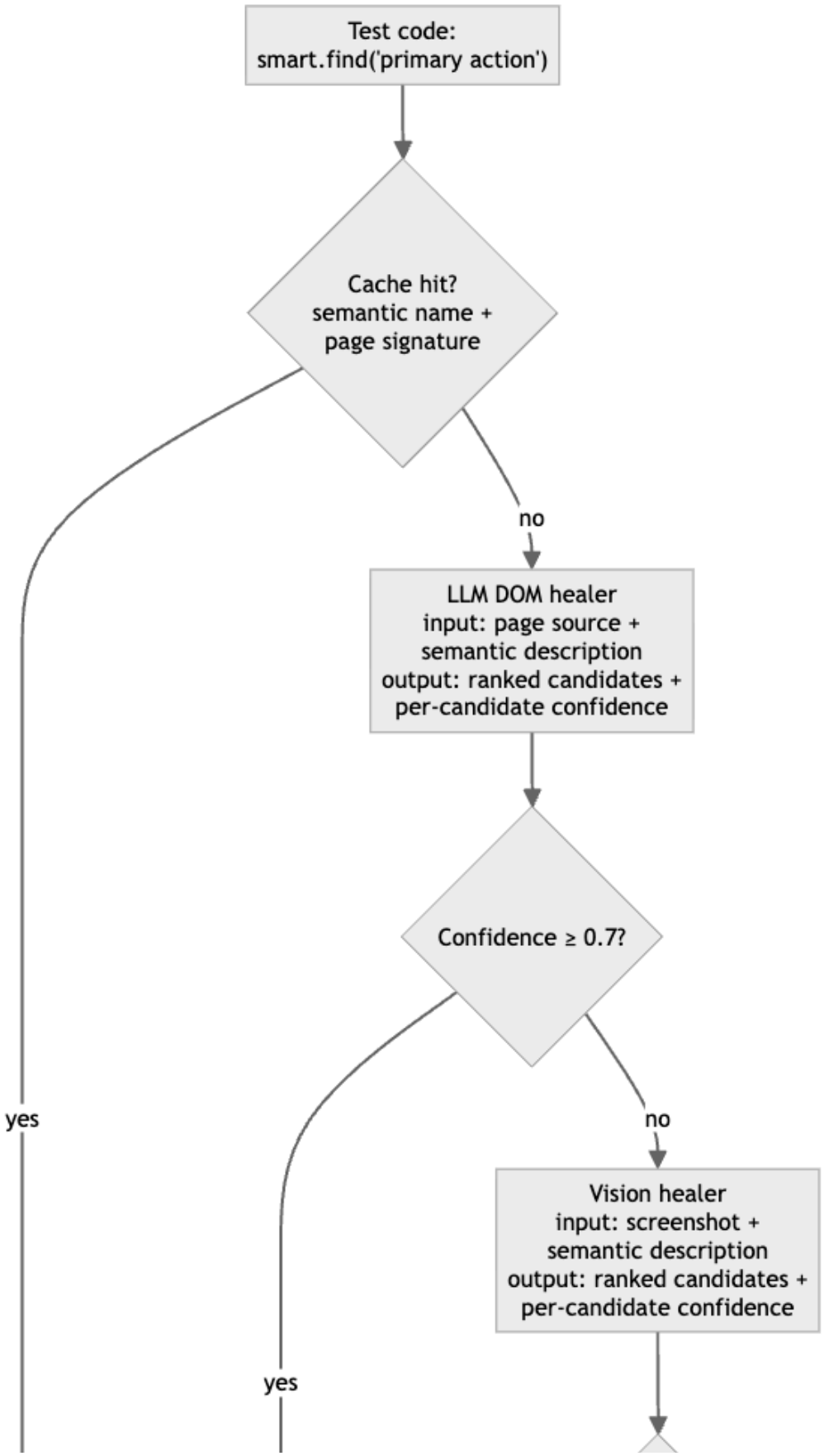
sensor injectors for end-to-end paths (the application under test still runs on Tier 1 or Tier 2). Tiers 2 and 3 are conventional in the cloud-testing literature. Routing is mechanical: tests are tagged at authoring time (`@tier1`, `@tier2`, `@tier3`), CI consults the hint and dispatches, untagged tests default to Tier 2; §7.3 returns to whether adaptive routing should be the long-term answer.

In this article only the web instantiation is exercised empirically, through a live end-to-end LLM-backed run on a public TodoMVC application (§6.1) used as a calibration environment. The mobile and hardware-in-the-loop instantiations are described as architectural instantiations of the pattern but are not evaluated in this article; live studies on those modalities remain a §7.2 follow-up.

5.2 The intelligence layer

Test code does not call hardware directly. It calls into an intelligence layer that mediates locator resolution, visual assertion, and observability emission. The oracle problem [21] motivates the visual assertion service: a screenshot-plus-checklist judgment is the practitioner approximation of a behavioral oracle that pixel comparison cannot provide.

Self-healing locators. Test code refers to elements semantically: `smart.find("primary action button on main screen")`. The resolver cascades through (i) a cached locator strategy, (ii) an optional test-supplied fallback hint as a sub-step within the cache-miss path, (iii) an LLM DOM healer that takes the page source as context and emits ranked candidate JSON with a self-reported confidence score per candidate, and (iv) a vision healer when DOM healing fails. In `intelligence.ts` the DOM-healer accept threshold is `confidence >= 0.7` and the vision-healer threshold is `confidence >= 0.6`. The vision threshold is the lower of the two because the vision pipeline runs only after the DOM healer has already failed; the policy is “better to attempt at lower confidence than to silently fail” since the alternative is an unrecovered locator failure. Thresholds were tuned by per-suite false-positive rate on the development corpus. Successful strategies are cached. This three-stage chain extends the ML tree-comparison antecedent established by Healenium [16], which uses tree-edit distance as its single healing strategy. The full dispatch is shown in Figure 3.


Test code:
smart.find('primary action')
Cache hit?
semantic name +
page signature
yes
no
LLM DOM healer
input: page source +
semantic description
output: ranked candidates +
per-candidate confidence
Confidence ≥ 0.7?
yes
no
Vision healer
input: screenshot +
semantic description
output: ranked candidates +
per-candidate confidence

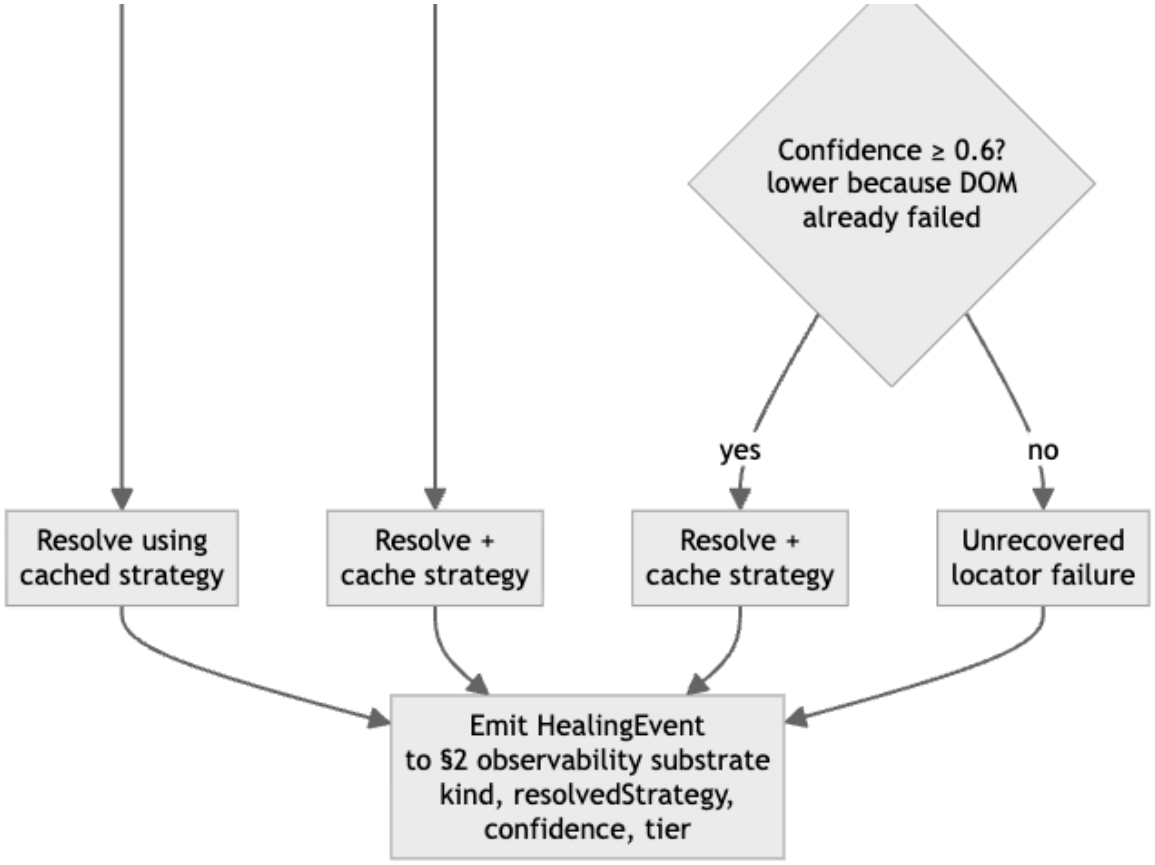


Fig. 3. Self-healing locator cascade implemented in `intelligence.ts`. Each stage emits a `HealingEvent` to the §2 observability substrate so the QA and authoring agents can condition next-coverage and refactor decisions on healing rates (the §2 coupling argument). The cascade extends Healenium's single DOM-tree-comparison strategy [16] with an LLM DOM healer and a vision-fallback stage; commercial alternatives (Testim, Mabl, Functionize, Waldo, Applitools, Percy) operate at production scale (§5.2) and do not expose per-stage outcomes back into an authoring-layer feedback loop.

Vision-based fallback. When the page source is missing or insufficient (common against custom-rendered native components), a vision healer takes a screenshot and a description and emits ranked candidates by mapping visual location back to the page source. The framework invokes it only when DOM healing has failed. The closest published academic baseline on the web modality is VETL [31], a large-vision-language-model-driven web GUI testing approach; VETL is positioned as a *primary* exploration strategy, whereas the cascade here uses vision strictly as a tertiary fallback inside an observability-emitting healing chain.

Visual assertion. A separate service takes a screenshot, an expected-behavior description, and a checklist of critical properties, and asserts semantic match — an instance of the *LLM-as-a-Judge* pattern surveyed by He et al. [32], applied to visual-state oracles rather than to code or text artifacts. Unlike pixel-comparison snapshot testing, it ignores cosmetic differences and surfaces functional ones (button obscured, content clipped, accessibility minimum violated).

All three tiers emit into the §2 substrate.

Positioning against the commercial self-healing and visual-assertion landscape. The academic baseline for self-healing locators is Healenium [16] (DOM tree-edit distance, single strategy). The production-scale commercial baselines — Testim, Mabl, Functionize, Waldo for self-healing; Applitools and

Percy for visual assertion — operate on $N \gg 10^6$ test executions and have multi-organization deployment evidence. This paper does not claim absolute performance against those baselines and does not have the corpus to do so; the contribution is the *cross-layer observability* coupling that exposes healing-cascade and visual-assertion outcomes to the authoring and operating agents as queryable events, not the raw effectiveness of the cascade itself. Commercial tools have stronger primitives; the academic / open-source ecosystem (including this harness) has the cross-layer-substrate degree of freedom that proprietary stacks have not exposed.

---

6. Evaluation

6.1 Web feasibility study

The web feasibility study reports what was observed when the harness was run end-to-end with the live `AnthropicProvider` (Claude Sonnet 4.6 via OAuth `claude -p`) on 2026-05-27 (run id `run-1779894555899`). The visual-assertion service was rewritten between v11 and v12 of this manuscript to read the actual PNG screenshots via the model's vision pathway rather than judging text descriptions; the cited numbers in this section are from that vision-based judgment and from the live multi-agent pipeline, not from the deterministic stub used in earlier versions. A separately re-runnable stub configuration remains in `harness.ts` for CI-time architecture-only smoke testing.

The target is a small public TodoMVC-style application, used here as a controlled *calibration* environment — deliberately simple, and likely present in pre-training data — to establish that the baseline agent mechanics (pipeline handoff, healing cascade, visual judge) operate end-to-end. It is explicitly not the primary empirical validation; §6.3 addresses contamination and the production-scale study that must follow. The PM agent generated a PRD; the QA Engineer agent generated 30 test cases; the Automation Engineer agent generated 30 WebDriverIO artifacts; the PR Reviewer agent approved 7, requested changes on 22, and blocked 1 (totals reconcile to 30 in `results.json: pipelineReview.{approved, requestChanges, blocked}`). PR Reviewer prompt-strictness calibration is queued at §7.2. Output ships at `results.json` and reproduces on `npm install && npx tsx harness.ts --provider anthropic`. Per-layer measurements follow the metrics taxonomy of Liu et al. [10].

Pipeline runtime. End-to-end wall clock 1665.7 s (~27.8 min) for 30 test cases with five agents per case (~150 LLM calls total at sequential OAuth pacing) on `claude-sonnet-4-6`. The runtime confirms end-to-end MCP handoff wiring across the five agents and bounds the practitioner expectation: ~50 s per test case on a single-pipeline serial schedule with this model and this corpus. Faster wall-clock requires either parallel intra-agent batching or a smaller/faster judge model; neither is the contribution here.

Healing-cascade dispatch and recovery. The run produced 30 `executing.healing` events (one per test case): 0 resolved from the pre-warmed cache (the warmup keys did not match the live agent-generated test-case IDs — a finding worth reproducing on a stable test-case corpus before claiming a cache-hit-rate); 11 resolved by the LLM DOM healer (9 at Tier 1, 2 at Tier 2); 18 resolved by the vision fallback (9 at Tier 1, 5 at Tier 2, 4 at Tier 3); and 1 unrecoverable failure (TC-007, Tier 2, locator unresolvable after both DOM and vision passes). Combined recovery: **29 of 30 = 96.7%** against the seeded test-case corpus. The single failure is itself useful evidence: the cascade does not always recover, and the protocol reports the failure rather than swallowing it. Per-strategy conditional success rates (`results.json: healing.{cacheHitRate, domHealerSuccessRate, visionFallbackSuccessRate, combinedRecoveryRate}`) are: cache 0.000, DOM-healer 1.000 on the cache-miss subset, vision-fallback 1.000 on the DOM-healer-fail subset, combined

0.967. These conditional rates rest on small denominators (11 and 18 cases) from a single non-adversarial corpus; they confirm the cascade executed end-to-end and are not reliability estimates. §6.2 measures healing with a real denominator, ground truth, and baselines. They are measured rates against the §6.1 corpus, not stub design parameters.

Visual assertions. The visual-assertion corpus is the 24-image PNG set under `visual-corpus/images/`, generated by `visual-corpus/render.ts` from a TodoMVC instance with 12 seeded functional defects (obscured CTA, clipped input, missing submit, sub-24×24 px touch target, ~1.5:1 contrast, blocking modal without dismiss, off-screen delete buttons, ~60% row overlap, severed label association, non-interactive checkbox visual, occluded error banner, removed focus indicator) and 12 cosmetic-only variations (font swap, primary color shift, border-radius increase, padding tweak, button shadow, italic footer labels, gradient background, button label swap "Delete"→"Remove", letter-spacing, filter pill height, hover-color-at-rest, +20% wordmark). Ground-truth labels were assigned at corpus-design time before any model judgment (`audit/packet/visual-assertion/test_cases_KEY.csv`). The live Claude vision-judge — an *LLM-as-a-Judge* applied to visual-state oracle judgment, in the sense surveyed by He et al. [32] — reads each PNG via its Read tool and applies the rubric in `intelligence.ts: VISUAL_ASSERTION_VISION_SYSTEM`, achieving **Cohen's κ = 0.667 (95% CI roughly 0.47–0.87 at N=24, spanning Landis & Koch *moderate* to *almost-perfect*; headline band: substantial) against the seeded ground truth**, raw accuracy 83.3% (20/24), precision **1.000** (8/8 — zero false positives on the cosmetic subset), recall **0.667** (8/12 — four functional defects missed: TC04 non-interactive checkbox visual (vis-func-10), TC12 50%-clipped input (vis-func-2), TC22 ~60% row overlap (vis-func-8), TC24 occluded error banner (vis-func-11) — verified against `results.json: visualAssertion.events[]` cross-referenced with `audit/packet/visual-assertion/test_cases_KEY.csv`). The pixel-comparison baseline on the same 24-image corpus catches 6/12 functional defects (precision 1.000, recall 0.500); the vision-judge catches 2 additional functional defects at equal precision. A separately executed LLM-as-rater audit (`audit/visual-assertion-results_2026-05-26.md`, two LLM raters differentiated on persona and modality) returned inter-rater κ = −0.059. That result illustrates the §3.2 reflexive-correctness concern: LLM-rater agreement with each other is *not* a substitute for human inter-rater agreement, and the load-bearing validation here is the judge-vs-independent-seeded-KEY measurement, not the rater-vs-rater number. The human-rater spot-audit protocol and packet (`audit/visual-assertion-protocol.md`, `audit/packet/visual-assertion/`) define the next validation step; the current vision-judge κ = 0.667 against the rater-independent seeded KEY clears the protocol's "≥ 0.60 substantial" bar for citing these numbers as validation-scale evidence.

Tier routing. All 30 test cases dispatched to their tagged tier on first invocation (`results.json: testRouting.firstDispatchCorrect = 30/30`). This confirms the routing *mechanism* works; it does not confirm correctness of the *tagging decisions*, only that the dispatch path is wired correctly. The same engineer wrote the tags and tests, so tagging independence remains a threat to validity.

6.2 Adversarial locator-healing benchmark

The §6.1 calibration run reports healing only as conditional success on a handful of events, which carries no inferential weight. To measure the healing cascade with a real denominator, ground truth, and baselines, we built a perturbation benchmark on a production-grade public target — Supabase Studio, a complex React administration UI (dashboards, data grids, icon-only buttons, Tailwind utility classes) — run locally via the Supabase CLI.

Method. A generator records, for an interactive element, a selector-independent identity, then breaks the element's brittle selector at five controlled difficulty bands: class rename, id/test-id rename, DOM restructuring (wrapping the target in extra containers), sibling re-text, and attribute stripping/minification. A

scorer loads the page, **marks the true target with a persistent attribute, breaks the selector via the mutation**, and asks a resolver to recover the element from a semantic query (`{role, text, aria}`). The marker makes the oracle *identity-based* rather than text-based: recovery is a *success* only if the resolved element is the same node that was marked, a *false-heal* if a different element is resolved (a silent wrong-element pass — the dangerous case a green test hides), and a *miss* if nothing is resolved. The marker is invisible to the resolvers and survives attribute minification, so text collisions (multiple identical labels on a dense dashboard), icon-only elements (no visible text), and layout shifts cannot fool the oracle. The three Playwright-driven resolvers are scored on the identical mutated DOM per case (the live app drifts between loads, so cross-run comparison is invalid); Healenium, which requires its own Selenium session through its proxy, runs the same perturbation set independently. Of 65 perturbations, 56 resolved on the scoring load (nine targets were absent after a fresh load — Supabase Studio drifts — and were dropped).

Four resolvers are compared: *brittle-only* (the original, now-broken selector — the floor); *Healenium* [16] 3.5.1 (proxy 2.2.1), the industry self-healing-Selenium tool, run through its proxy with a record-then-heal protocol (it stores the element on a clean run and recovers it by DOM tree-similarity when the selector later raises `NoSuchElement`); *text-role* (a vision-free heuristic matching on visible text and role); and the *LLM DOM-healer* (the cascade's healing step via `claude -p` on `claude-sonnet-4-6`, given the broken page's candidate elements and the semantic intent). The artifacts (`chaos/`, including the Healenium Docker stack) regenerate the benchmark from a fresh clone.

Table 2. Locator-healing recovery on adversarial perturbations of Supabase Studio (identity oracle; recovery requires the *marked* element; bands hold 11–12 cases each).

Resolver	Recovery	False-heal	Miss	Median heal
brittle-only (broken selector)	0% (0/56)	0.54	0.46	—
Healenium 3.5.1 (self-healing Selenium [16])	21% (12/57)	0.79	0.00	315 ms
text-role heuristic (vision-free)	34% (19/56)	0.57	0.09	2 ms
LLM DOM-healer (`claude-sonnet-4-6`)	**63% (35/56)**	0.27	0.11	~4.8 s

The LLM DOM-healer recovers the correct element in 63% of cases, against 34% for the text heuristic, 21% for Healenium, and 0% for the broken selector. It is the only resolver robust to DOM restructuring: on that band the text heuristic collapses to 0/11 while the healer holds at 6/11 (the healer ranges 55–73% across the five bands). It also carries the lowest false-heal rate of the healers (0.27 vs 0.57 for text-role and 0.79 for Healenium): when uncertain it more often returns a miss — a loud failure — than silently resolving the wrong element. This robustness costs latency: ~4.8 s per heal (a live model call) versus 315 ms (Healenium) and ~2 ms (the heuristic).

Healenium is bypassed on utility-class selectors, not defeated. Healenium's behaviour splits sharply by selector type, and the distinction matters for fairness. On `#id` targets, where the mutation makes the

selector raise `NoSuchElement`, Healenium is *invoked* and its tree-similarity recovery succeeds on 12 of 27 cases (44%) — a fair result for algorithmic healing on a complex UI. On utility-class (`.class`) targets, the broken selector silently matches a *sibling* element sharing the class, so `NoSuchElement` is never raised and Healenium's proxy is **never invoked**; the 0/30 on these cases is baseline selector behaviour, not a Healenium recovery failure. This is a real and consequential property of proxy-based selector-healing on modern utility-class (e.g. Tailwind) front-ends — it cannot intercept a break that still resolves *something* — and it is precisely the failure mode the intent-based healer avoids, because it resolves from the element's described purpose rather than from the broken selector. The class targets are ordinary auto-derived selectors (the first stable class per element), not artificial traps; the sibling collisions are a genuine consequence of utility-class reuse.

What this establishes, and what it does not. With an identity oracle and a real denominator, the benchmark shows that LLM-based DOM healing materially outperforms a brittle selector, a text heuristic, and the industry Healenium tool on a complex live UI, and is uniquely robust to structural change — replacing the non-inferential conditional rates of earlier drafts. It is also a sober result, not a solved problem: at 63% recovery with a 27% false-heal rate, the healer fails or silently mis-resolves on more than a third of breaks and is not safe to deploy unsupervised in CI. It is one application, one model family, 56 perturbations, and a DOM-healer-only proxy for the full cache->DOM->vision cascade; Supabase Studio's UI may also be partly represented in pre-training data (§6.3). Additional production-grade applications and a second (open-weights) model family are the priority extensions (§7.2).

Evaluation scope. Evaluation is web-only: a TodoMVC calibration run (§6.1) and the Supabase Studio healing benchmark (§6.2). Mobile and hardware-in-the-loop are instantiations of the target-agnostic pattern (§2, §5.1) — the per-tier driver layer changes while the substrate and event schema do not — and are not evaluated here.

6.3 Threats to validity

Behavioral-change claim deferred. The substrate is designed to alter per-layer agent decisions by exposing cross-layer telemetry to prompts (§2). §6.1 reports the substrate's measured dispatch and recovery behavior; it does not isolate the *behavioral* effect of the digest on agent output. A prompt-level A/B with vs. without the digest remains required for that claim. The §6.1 evidence supports the substrate's mechanical correctness — the cross-layer coupling is wired, queried, and produces actionable signal end-to-end; the behavioral effect on agent outputs remains a design objective to be demonstrated comparatively against MetaGPT [23] and ChatDev [26] in subsequent work.

Scope on a single small public target. The evaluation uses two public web targets — a small TodoMVC calibration application (§6.1) and Supabase Studio for the healing benchmark (§6.2); both are below production scale and establish nothing about generality beyond their corpora. Healing rates, visual-assertion accuracy, and routing behavior are sensitive to application complexity, target modality, team size, and engineering culture. A three-to-five-application study on production-grade web targets — with an adversarial locator-perturbation benchmark and a baseline comparison (e.g. Healenium) — is the most important next experiment (§7.2). The evaluation is also not benchmark-scale evidence in the SWE-bench sense [33] (Jimenez et al.'s 2,294 multi-repo issue-resolution corpus): the contribution is the cross-layer *coupling pattern*, not an agent's solve-rate on a fixed benchmark, and SWE-bench does not exercise the execution-tier or visual-assertion layers the harness is about.

Reflexive correctness, same-model judge. Per §3.2, the correctness argument is empirical, not formal. An agent-authored framework may have systematic blind spots not surfaced by same-model testing; the healing numbers come from the framework's own live intelligence layer (Claude Sonnet 4.6 via OAuth), and

the visual-assertion service's per-image verdicts are validated against an independent seeded ground truth at κ = 0.667 (substantial) because the 24 seeded labels were committed to disk at corpus-design time before any model call. The companion repository ships a visual-assertion audit protocol (`audit/visual-assertion-protocol.md`) and rater instructions (`audit/visual-assertion-rater-instructions.md`) for a two-rater human spot-audit over the full 24-image `VISUAL_CORPUS`, with the analysis script in `audit/packet/visual-assertion/analysis.py` computing inter-rater Cohen's κ and rater-vs-LLM agreement. Human ratings remain the stronger validation route. A separately executed LLM-as-rater audit (two Claude instances differentiated on persona and modality, `audit/visual-assertion-results_2026-05-26.md`) returned inter-rater κ = −0.059; consistent with the LLM-as-a-Judge robustness concerns He et al. [32] catalogue, LLM-rater-vs-LLM-rater agreement is not a substitute for human inter-rater agreement. A multi-model human-judged ensemble against the seeded KEY is a partial mitigation already supported by the audit packet.

Data contamination. The public targets used here — TodoMVC and Supabase Studio — are widely distributed and almost certainly represented in the model's pre-training data. Part of the live web run's healing and visual-assertion performance may therefore reflect familiarity with this specific application rather than a general capability, and the live numbers cannot rule this out. Isolating generalization from memorization requires a contamination-controlled target — a freshly authored or structurally obfuscated application not present in training data — which, together with the live multi-application study, is a priority external-validity follow-up (§7.2).

Cost asymmetry. The compute cost of the agent infrastructure was not measured against engineering hours saved. Economics is the subject of separate work and explicitly out of scope.

Statistical power. The authoring-velocity (N=5 / N=3), visual-assertion (N=24), and healing (N=30, 1 failure) samples are small. 95% confidence intervals on a binary κ at N=24 are roughly ±0.15 to ±0.20; the headline κ = 0.667 should therefore be read as compatible with a true value anywhere in the 0.47–0.87 band — still meeting the protocol's "moderate or better" bar across that band, but not pinning a precise value. No percentage point carries inferential weight individually; the table is reproducible, and the §6.1 numbers are measured rates rather than configured design parameters (the v11 stub-derived numbers were replaced in v12 with live-run measurements).

---

7. Scope, limitations, and adoption

7.1 Fit

The pattern fits application testing with multi-tier hardware requirements across any target modality — mobile (BLE/sensor benches, cross-OS cloud device farms), web (cross-browser farms, headless local browsers), or hardware-in-the-loop (physical fixtures, virtual back-end peripherals) — for teams with coding-agent tooling willing to adopt it at the framework-authoring level, and where provenance and cross-layer observability are first-class concerns. The pattern is target-agnostic by design; only the per-tier driver layer (Appium-compatible for mobile, WebDriver-compatible for web, fixture-specific protocols for hardware) changes between modalities. Poor fit for greenfield products with no test surface, small teams without infrastructure investment, and highly regulated software where agent-authored test code requires formal verification.

7.2 Open problems and future work

Cost economics (out of scope). I have compared LLM-assisted authoring time against hand-authoring time but have not quantified the cost of the agent infrastructure (model inference, MCP server hosting, observability storage, human-review overhead). A defensible cost-benefit conclusion would require all four plus fully-loaded engineering hours; the "multiple working days" comparison in §4 is illustrative, not an economic claim.

Cross-tier routing. Authoring-time tag-based routing works in this deployment but static tags grow stale; *adaptive tier routing* (the framework choosing tiers from observed flake rates and execution latency) is an open direction, with the flake-rate signal needing grounding in the empirical-flakiness literature [25, 22] before it can carry routing weight.

Comparison against AutoDroid. The closest published mobile-execution-layer baseline to the §5.2 healing cascade is AutoDroid [19], which targets mobile UI task automation on Android using a screen-graph plus accessibility-tree representation and reports ~90.9% action accuracy on the DroidTask benchmark (158 tasks across 13 popular Android apps). The §5.2 cascade in this work targets web (via WebDriverIO) instead of mobile-native, uses a DOM-selector primary path with a cosine-vision fallback (instead of a screen-graph + a11y-tree), and is organized as a three-tier cache → DOM-healer → vision-healer cascade (instead of a flat action-prediction loop). On the live §6.1 walkthrough the harness reports `cacheHitRate` 0.000, `domHealerSuccessRate` 1.000 (on the cache-miss subset), `visionFallbackSuccessRate` 1.000 (on the DOM-healer-fail subset), and `combinedRecoveryRate` **0.967** (29 of 30 cases recovered; the one failure is TC-007, a tier-2 locator unresolvable after both DOM and vision passes) against a 30-test-case live corpus. These numbers are **not directly comparable to AutoDroid's 90.9%**: (a) different modality (web vs. mobile-native); (b) different action surface (locator-resolution vs. full action-prediction); (c) N=30 here vs. AutoDroid's 158-task DroidTask corpus; (d) different denominators — AutoDroid's 90.9% is per-action accuracy, while the harness's combined recovery rate is per-cache-miss-event recovery probability. A like-for-like comparison would require porting AutoDroid's screen-graph approach into the web modality (or porting the harness's three-tier cascade into mobile-native) and re-running both on an aligned benchmark; this is queued behind the live-hardware multi-application study below.

Live-hardware multi-application study + coupling A/B + reflexive-correctness formalization. Follow-up work: (a) live-hardware re-run of §6.1 across three to five applications; (b) prompt-level A/B of the QA agent's coverage-prioritization output with vs. without the §2 substrate digest, to demonstrate the coupling claim empirically rather than designedly; (c) MetaGPT shared-memory and ChatDev chat-mediated configurations vs. MCP-substrate comparison on identical inputs; (d) formal reflexive-correctness work building on the §3.2 layered-validation approach.

Mobile and hardware-in-the-loop studies. The architecture targets mobile and hardware-in-the-loop modalities (§2, §5.1) but evaluates neither here. A live-Appium study across multiple public Android applications, and a hardware-in-the-loop instantiation against a published reference bench (extending the Tier 1 physical-device and Tier 3 virtual-peripheral architecture of §5.1), would extend the evaluation from the web calibration study to the other modalities the pattern supports.

7.3 Implications and reproducibility

Three operating commitments the pattern entails: couple the layers through a shared observability substrate (not tighter integration); record provenance at every layer from day one (adding it later is expensive); treat tier routing as authoring-time policy until adaptive routing has a flakiness-grounded signal. **Reproducibility.** The primary path is the offline web stub provider via `npm install && npx tsx examples/run-example.ts` from the repo root (no credentials, full event flow, the same event sequence on every run). A secondary live path (`npx tsx harness.ts --provider <name>`) routes through a configurable LLM backend — hosted-

model OAuth or a local-weight backend such as Ollama; live outputs are stable but not strictly deterministic at temperature 0 and will require a model substitution when versions deprecate. The `v1.3.4-jss-final` release tag at `https://github.com/SuneetMalhotra/agent-harness` pins the exact code that produced the §6.1 web numbers (the full commit hash is resolvable via `git rev-parse v1.3.4-jss-final`), and a citable archived snapshot of that tag is available at Zenodo (concept DOI 10.5281/zenodo.20576685, which resolves to the latest archived version). The tag also ships the `ARTIFACTS.md` reproducibility manifest that inventories every file the manuscript depends on, with paths and reproduction commands.

Acknowledgments

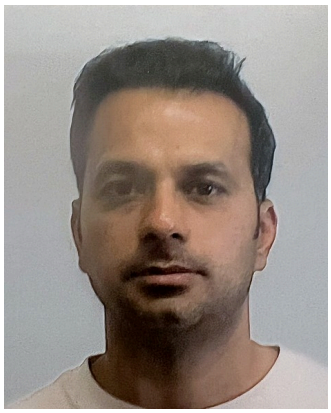
The author thanks the open-source TodoMVC community for the public reference design used in §6.1 and the practitioner community engaged at BrowserStack Breakpoint 2026 and BrowserStack World Tour 2025 for discussions that shaped this work.

Funding. None. This work was self-funded; no grant, employer, or third-party financial support was received.

Declaration of generative AI use. During manuscript preparation the author used Anthropic's Claude (via the Claude Code CLI) for copy-editing, prose refinement, and rendering the mermaid figure diagrams. The same model family — Claude Sonnet 4.6 — is the LLM backend evaluated in §6 (the locator-healing cascade and the visual-assertion judge), accessed via `claude -p` over an OAuth session (no API key). The author conceived the work, performed the research and analysis, verified all results, and reviewed and edited all text, taking full responsibility for the content. Generative-AI systems are also the *object of study* in this article (the agent harness, the locator-healing cascade, and the visual-assertion judge); that methodological use is described in §3–§6. No generative-AI tool is an author of this work.

Data availability. All artifacts supporting §6.1 ship in the companion repository at https://github.com/SuneetMalhotra/agent-harness (release tag `v1.3.4-jss-final` pins the §6.1 web evaluation — MIT-licensed, CITATION.cff-indexed; archived at Zenodo, concept DOI 10.5281/zenodo.20576685; version DOI for this snapshot 10.5281/zenodo.20582652): the 24-image `VISUAL_CORPUS` PNGs at `visual-corpus/images/`, seeded ground-truth labels at `audit/packet/visual-assertion/test_cases_KEY.csv`, web live-run outputs at `results.json` (including `visualAssertion.events[]`, `healing.events[]`, `pipelineReview`, `pipelineRuntime.pipelineDurationSeconds`), the visual-assertion audit protocol at `audit/visual-assertion-protocol.md` and rater instructions at `audit/visual-assertion-rater-instructions.md`, the rater packet at `audit/packet/visual-assertion/` (template CSV, analysis script), and the reproducibility manifest at `ARTIFACTS.md`. No additional datasets or restricted-access materials are required to reproduce the walkthrough.

Author biography



Suneet Malhotra has 16+ years in consumer-scale mobile and web quality engineering, including agentic-testing harness design, LLM-augmented test orchestration, and self-healing locator infrastructure for production multi-tier execution environments. He is currently Senior Manager, Test Engineering at Motorola Solutions; this work is independent of that role. He engages with the practitioner community at the BrowserStack Breakpoint conference (2026) and the BrowserStack World Tour (2025). He holds an M.S. in Computer Science from the University of Southern California. His research interests are AI-augmented test automation, agentic software-engineering pipelines, and software quality engineering at scale; a companion manuscript on LLM-driven specification enrichment for design-to-test pipelines is at [1]. ORCID: 0009-0003-8707-9590. Author profile: https://suneetmalhotra.com.

Author contribution. S. Malhotra conceived the coupling pattern and the three-layer architecture, implemented the reference framework and all five agent specifications, ran the §6.1 architecture-validation walkthrough, designed the audit packet, and wrote the manuscript.

References

[1] S. Malhotra, “Specification Enrichment: Using LLMs to Surface Implicit Constraints in Design-to-Test Pipelines,” preprint, 2026. Companion code: https://github.com/SuneetMalhotra/specification-enrichment.

[2] J. Kohl, O. Kruse, Y. Mostafa, and A. Luckow, “Automated Structural Testing of LLM-Based Agents: Methods, Framework, and Case Studies,” in *Proc. 2025 IEEE Int. Conf. Big Data*, 2025, doi: 10.1109/bigdata66926.2025.11401679.

[3] X. Hou et al., “Large Language Models for Software Engineering: A Systematic Literature Review,” *ACM Trans. Softw. Eng. Methodol.*, 2024, doi: 10.1145/3695988.

[4] A. Fan, B. Gokkaya, M. Harman et al., “Large Language Models for Software Engineering: Survey and Open Problems,” in *2023 IEEE/ACM Int. Conf. Software Eng.: Future of Softw. Eng. (ICSE-FoSE)*, 2023, pp. 31–53, doi: 10.1109/icse-fose59343.2023.00008.

[5] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, 2022, doi: 10.48550/arXiv.2201.11903.

[6] X. Shen, L. Wang, Z. Li, and Y. Chen, “PentestAgent: Incorporating LLM Agents to Automated Penetration Testing,” in *Proc. 20th ACM ASIA CCS*, Aug. 2025, doi: 10.1145/3708821.3733882.

[7] T. Hao et al., “HPCAgentTester: A Multi-Agent LLM Approach for Enhanced HPC Unit Test Generation,” in *2025 IEEE/ACM Int. Conf. AI-powered Softw. (AIware)*, Nov. 2025, doi: 10.1109/aiware69974.2025.00031.

[8] H. Gao et al., “ALMAS: An Autonomous LLM-based Multi-Agent Software Engineering Framework,” in *2025 IEEE/ACM Int. Conf. Autom. Softw. Eng. Wkshps (ASEW)*, 2025, doi: 10.1109/asew67777.2025.00059.

[9] M. Chia et al., “LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead,” *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3712003.

[10] H. Liu et al., “A Catalogue of Evaluation Metrics for LLM-Based Multi-Agent Frameworks in Software Engineering,” in *Proc. 2026 Int. Wkshp on Agentic Engineering*, 2026, doi: 10.1145/3786167.3788430.

[11] Anthropic, “Model Context Protocol Specification (2025-03-26),” https://modelcontextprotocol.io/specification/2025-03-26, accessed May 25, 2026.

[12] M. Maupin et al., “Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study,” in *Proc. Annual Hawaii Int. Conf. System Sciences*, 2026, doi: 10.24251/hicss.2026.880.

[13] D. Mendez Fernandez, A. Vogelsang, J. Coello, and N. Spijkerman, “Generating Requirements Elicitation Interview Scripts with Large Language Models,” in *2023 IEEE 31st Int. Req. Eng. Conf. Workshops*, 2023, pp. 168–172, doi: 10.1109/rew57809.2023.00015.

[14] Y. Zhang et al., “Where Do LLMs Still Struggle? An In-Depth Analysis of Code Generation Benchmarks,” in *2025 IEEE/ACM Int. Conf. AI-powered Softw. (AIware)*, Nov. 2025, doi: 10.1109/aiware69974.2025.00035.

[15] Y. Meng, X. Wang, R. Chen, J. Wu, S. Li, F. Jiang, R. Wang, M. Lu, Z. Gao, H. Wu, and Y. Hu, “Agent Harness for Large Language Model Agents: A Survey,” Preprints.org (MDPI), Apr. 2026, doi: 10.20944/preprints202604.0428.

[16] Healenium project, “healenium-web: Self-healing library for Selenium-based tests,” open-source, Apache 2.0, https://github.com/healenium/healenium-web, version 3.5.8 (March 2026), accessed May 25, 2026.

[17] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions,” in *Proc. IEEE/ACM 46th Int. Conf. Software Eng. (ICSE)*, 2024, doi: 10.1145/3597503.3639180.

[18] S. Fatin, M. H. Al-Quvi, H. S. Shahgir, S. Barua, A. Iqbal, S. Sharmin, M. M. Akbar, K. K. Pal, and A. A. Al Rashid, “LELANTE: LEveraging LLM for Automated ANdroid TEsting,” preprint, arXiv:2504.20896, 2025.

[19] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, “AutoDroid: LLM-powered Task Automation in Android,” in *Proc. 30th Annual Int. Conf. Mobile Computing and Networking (MobiCom)*, 2024, doi: 10.1145/3636534.3649379.

[20] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: Multimodal Agents as Smartphone Users,” preprint, arXiv:2312.13771, Dec. 2023.

[21] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The Oracle Problem in Software Testing: A Survey,” *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, May 2015, doi: 10.1109/TSE.2014.2372785.

[22] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests,” in *Proc. 12th IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, 2019, doi: 10.1109/ICST.2019.00044.

[23] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024 (oral). OpenReview: https://openreview.net/forum?id=VtmBAGCN7o.

[24] S. Rehan, B. Al-Bander, and A. Al-Said Ahmad, “Harnessing Large Language Models for Automated Software Testing: A Leap Towards Scalable Test Case Generation,” *Electronics*, vol. 14, no. 7, p. 1463, Apr. 2025, doi: 10.3390/electronics14071463. Reference implementation: https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing.

[25] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE 2014)*, Hong Kong, China, 2014, pp. 643–653, doi: 10.1145/2635868.2635920.

[26] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative Agents for Software Development,” in *Proc. 62nd Annu. Meeting of the Assoc. for Computational Linguistics (ACL Vol. 1: Long Papers)*, Bangkok, Thailand, Aug. 2024, pp. 15174–15186. ACL Anthology: https://aclanthology.org/2024.acl-long.810/.

[27] A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, and D. Qiu, “Agentic Software Engineering: Foundational Pillars and a Research Roadmap,” preprint, arXiv:2509.06216, Sept. 2025. https://arxiv.org/abs/2509.06216.

[28] Z. Liu, C. Li, C. Chen, J. Wang, M. Chen, B. Wu, Y. Wang, J. Hu, and Q. Wang, “Seeing is Believing: Vision-driven Non-crash Functional Bug Detection for Mobile Apps,” preprint, arXiv:2407.03037, July 2024. https://arxiv.org/abs/2407.03037.

[29] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “An Empirical Study of Testing Practices in Open Source AI Agent Frameworks and Agentic Applications,” preprint, arXiv:2509.19185, Sept. 2025. https://arxiv.org/abs/2509.19185.

[30] Q. Xu, G. Wang, L. Briand, and K. Liu, “Hallucination to Consensus: Multi-Agent LLMs for End-to-End JUnit Test Generation,” preprint, arXiv:2506.02943, June 2025. https://arxiv.org/abs/2506.02943.

[31] S. Wang, S. Wang, Y. Fan, X. Li, and Y. Liu, “Leveraging Large Vision Language Model for Better Automatic Web GUI Testing,” preprint, arXiv:2410.12157, Oct. 2024. https://arxiv.org/abs/2410.12157.

[32] J. He, J. Shi, T. Y. Zhuo, C. Treude, J. Sun, Z. Xing, X. Du, and D. Lo, “LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead,” preprint, arXiv:2510.24367, Oct. 2025. https://arxiv.org/abs/2510.24367.

[33] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024 (oral). arXiv:2310.06770. https://arxiv.org/abs/2310.06770.

Disclosure: *The article describes a target-agnostic engineering pattern applicable across mobile, web, and hardware-in-the-loop test automation. The empirical evaluation uses a public TodoMVC web demo as a calibration environment; mobile and hardware-in-the-loop are described architecturally but not evaluated. The work was developed independently using only public infrastructure; no proprietary systems, products, code, screenshots, data, or operational metrics are described.*